

PRUEBA TÉCNICA DESARROLLADOR MICROSERVICIOS

La siguiente prueba pretende medir los conocimientos técnicos sobre el manejo de las tecnologías que se están utilizando para el cargo al cual se encuentran postulando.

Para la prueba se cuenta con 10 horas, que empiezan a contar una vez recibida la prueba por lo cual el candidato debe contar con este espacio para poder enfocarse a realizar esta.

Esta prueba se debe enviar el comprimido del proyecto. Se debe enviar a los siguientes correos para su evaluación:

juliana.cardona.calderon@audifarma.com.co

julianh@audifarma.com.co

sergio.gallego.tovar@audifarma.com.co

ENUNCIADO

Se requiere crear 1 microservicio, para el manejo de cliente y sus direcciones que permita las operaciones de crear, modificar y consultar el cliente con sus direcciones. Un cliente puede tener múltiples direcciones, por lo tanto, se pueden agregar o eliminar direcciones de un cliente. Los campos necesarios para el modelo son:

Cliente	
nombre	String
apellido	String
numeroDocumento	String
tipoDocumento	String
Edad	Entero
Activo	Booleano

Direccion	
departamento	String
ciudad	String
direccionCompleta	String

OBJETIVO

Desarrollar un microservicio en Java listo para ser desplegado en un orquestador de contenedores como Kubernetes, aplicando buenas prácticas de arquitectura, código limpio y cultura DevOps.

REQUERIMIENTOS

1. Código Fuente

- Desarrollar el microservicio utilizando Spring Boot 3.x con Java 21.
- Aplicar Arquitectura Hexagonal (Ports & Adapters), separando claramente:
 - domain: lógica de negocio pura, sin dependencias de frameworks.
 - Application: casos de uso / servicios de aplicación.
 - Infrastructure: adaptadores de entrada (REST) y salida (persistencia, mensajería, etc.).
- Usar H2 Database en memoria como motor de persistencia, configurado desde application.yml.
- Exponer una API REST documentada con Springdoc OpenAPI (Swagger UI) (opcional pero valorado).

2. Calidad de Código

- Seguir los principios SOLID y Clean Code.
- Usar records de Java para DTOs y objetos de valor inmutables.
- Gestionar dependencias con Maven.

3. Pruebas

- Pruebas unitarias con JUnit 5 y Mockito cubriendo los casos de uso.
- Pruebas de integración con @SpringBootTest y MockMvc o WebTestClient para los endpoints REST.
- Generar reporte de cobertura >80% con JaCoCo (opcional pero valorado).

4. Containerización y Despliegue

- docker-compose.yml para levantar el entorno de desarrollo local.
- Manifiestos de Kubernetes (k8s/)
- Exponer el endpoint /actuator/health de Spring Boot Actuator como liveness y readiness probe en Kubernetes (opcional pero valorado).

5. README

El archivo README.md debe incluir:

- Descripción general del proyecto y su arquitectura.
- Diagrama o descripción de la estructura de paquetes.
- Requisitos previos (JDK, Docker, kubectl, etc.).
- Instrucciones para ejecutar localmente con Maven.
- Instrucciones para construir y ejecutar con Docker.
- Instrucciones para desplegar en Kubernetes.
- Colección de Postman o ejemplos curl para consumir la API.

NOTA:

Se permite el uso de herramientas de IA generativa (ChatGPT, GitHub Copilot, Claude, etc.) como apoyo durante el desarrollo. Pero la sustentación técnica tendrá el mismo peso evaluativo que la entrega del proyecto.